

Data Structures and Algorithms - 2025 - 2026

Home / My courses / IE - DSA / Weekly Quiz / Lecture 4 Quiz

Quiz navigation

1

2

3

4

5

6

7

8

9

10

11

12

13

14

Show one page at a time

Finish review

|           |                                |
|-----------|--------------------------------|
| Status    | Finished                       |
| Started   | Sunday, 29 March 2026, 3:49 PM |
| Completed | Sunday, 29 March 2026, 3:59 PM |
| Duration  | 10 mins 36 secs                |
| Marks     | 10.75/14.00                    |
| Grade     | 7.68 out of 10.00 (76.79%)     |

Question 1  
Correct  
Mark 1.00 out of 1.00  
Flag question

If we have a fixed-capacity Queue, which operations can throw an exception?

Select one or more:

- ☒ push
- ☒ pop
- ☒ top
- ☐ isEmpty
- ☐ isFull

Your answer is correct.

Pop and top can throw an exception for any Queue (fixed capacity or not) if the queue is empty.

But if we have a fixed capacity, we can have a full queue, and then push can also throw an exception.

Technically init can also throw an exception if the capacity is negative. But I did not add this option because this was not discussed separately.

The correct answers are: push, pop, top

Question 2  
Correct  
Mark 1.00 out of 1.00  
Flag question

Assume that we implement a Queue on a circular dynamic array. Which operation will have a  $\Theta(n)$  complexity in the worst case?

Select one or more:

- ☒ push
- ☐ pop
- ☐ top
- ☐ isEmpty
- ☐ all operations have  $\Theta(1)$  complexity in the worst case

Your answer is correct.

This is tricky. We use circular arrays to get better complexity for the operations than in case of a regular array. But, it is a dynamic array and we are talking about worst case time complexity and this can happen for push, if we need to do a resize. Since resize happens rarely (assuming correct implementation) push has a  $\Theta(1)$  amortized complexity in total, but the worst case is still  $\Theta(n)$ .

Optionally, pop could also have  $\Theta(n)$  worst case performance if we do a resize. But for pop we don't have to do resize.

The correct answer is: push

Question 3  
Correct  
Mark 1.00 out of 1.00  
Flag question

In a List there are positions and the interface of the List contains operations which take a position as parameter. All these operations require the position to be valid. For which operation is the definition of a valid position different from the definition considered by other operations?

Select one or more:

- ☐ getElement
- ☐ remove
- ☒ addToPosition
- ☐ setElement

Your answer is correct.

In general a position is valid if it represents an actual element from the list. For a list with 5 elements, valid positions are 1, 2, 3, 4, 5 (assume 1-based indexing). However, when we add a new element, it can be added at the end of the list (at position 6 in the above example), so for operation addToPosition we have a different definition of when is a position valid.

The correct answer is: addToPosition

Question 4  
Partially correct  
Mark 0.50 out of 1.00  
Flag question

What is the main difference between ADT IndexedList and ADT IteratedList?

Select one or more:

- ☒ IndexedList does not have an iterator, while IteratedList has.
- ☐ There is no difference between them, they denote the same container, but it can be found under different names in different books.
- ☒ The way in which positions are defined
- ☒ In an IteratedList there are no positions, while in IndexedList there are.
- ☐ IndexedList is implemented on a dynamic array, while IteratedList is implemented on a linked list

Your answer is partially correct.

You have selected too many options.

The most important property of ADT List is the existence of positions (it is the only container which has positions). The difference between IndexedList and IteratedList is how do we define the notion of a position: in case of IndexedList positions are represented by integer numbers, while in case of IteratedList positions are represented by iterators.

The correct answer is: The way in which positions are defined

Question 5  
Correct  
Mark 1.00 out of 1.00  
Flag question

Which of the following operations from the interface of ADT List no longer exist for ADT SortedList?

Select one or more:

- ☒ addToEnd
- ☐ removePosition
- ☐ getElement
- ☐ size
- ☒ setElement
- ☐ position
- ☐ search
- ☐ isEmpty

Your answer is correct.

In a SortedList elements are sorted, so operations like addToEnd make no sense (when an element needs to be added, there is one single position where it should be - according to the relation used for sorting - so you cannot force an element to be added to the end, it might mess up the sorting). Similarly, setElement might mess up the sorting of the elements as well.

The correct answers are: addToEnd, setElement

Question 6  
Correct  
Mark 1.00 out of 1.00  
Flag question

Which of the following containers does not have iterator?

Select one or more:

- ☐ Bag
- ☐ SortedBag
- ☐ Set
- ☐ SortedSet
- ☒ Stack
- ☒ Queue
- ☒ Priority Queue
- ☐ Map
- ☐ SortedMap
- ☐ MultiMap
- ☐ SortedMultiMap

Your answer is correct.

ADT Stack, Queue and PriorityQueue have so called restricted access to the elements, meaning that we cannot see their content, so they do not have iterator.

The correct answers are: Stack, Queue, Priority Queue

Question 7  
Correct  
Mark 1.00 out of 1.00  
Flag question

Which is the correct representation of a singly-linked list node?

Select one or more:

- ☐ Node:  
next:  $\uparrow$  Node
- ☐ Node:  
value: TElem
- ☐ Node:  
info: TElem  
next:  $\uparrow$  Node  
prev:  $\uparrow$  Node
- ☒ Node:  
info: TElem  
next:  $\uparrow$  Node

Your answer is correct.

In a singly linked list every node contains an information (of type TElem) and the address of the next node (pointer to a Node).

The correct answer is: Node:  
info: TElem  
next:  $\uparrow$  Node

Question 8  
Correct  
Mark 1.00 out of 1.00  
Flag question

For a singly linked list, in general, we store only the head of the list. What operation would have a better complexity if we stored the tail as well?

Select one or more:

- ☒ addToEnd
- ☐ removeEnd
- ☐ search
- ☐ size
- ☐ getElement

Your answer is correct.

For size, normally, you have to count every node.

For search you have to check every node, until we either find the element we are looking for, or we get to the end of the list.

For getElement we have to go through the nodes until we get to the required position. Storing the tail we would have access to the last element, but we don't know its position and even if we knew, this would be another best case, it would not change the overall complexity of the function.

For addToEnd having the tail would reduce the complexity to  $\Theta(1)$

For removeEnd having the tail is not useful, because removing the last element means setting the "next" of the previous element to NIL. But we do not have access to the previous element, and getting it means going through all the nodes of the list.

The correct answer is: addToEnd

Question 9  
Incorrect  
Mark 0.00 out of 1.00  
Flag question

Consider the following implementation of the search function for a singly linked list (assume the representation from Lecture 4), which returns the node with elem, or NIL if elem is not in the list.

```
function search(sll, elem) is:
    current ← sll.head
    while [current].info ≠ elem AND current ≠ NIL execute
        current ← [current].next
    end-while
    search ← current
end-function
```

Which of the following statements is/are correct?

Select one or more:

- ☐ The implementation is correct.
- ☒ The implementation will not work correctly if the list is empty, but for all other cases it will return the correct answer.
- ☐ The implementation will not work correctly if the element we are looking for is the last, but otherwise it works correctly
- ☐ The implementation will not work correctly if the element is not in the list, otherwise it works correctly.
- ☐ The implementation will not work correctly if the element is in the list, otherwise it works correctly.
- ☐ The implementation will not work correctly if the element appears multiple times in the list, otherwise it works correctly.

Your answer is incorrect.

Whenever we dereference a node (we access its content) we have to be sure that the node is not NIL. We check if current is NIL in the code, but only after dereferencing it. So if current is NIL, it is going to crash before the condition current ≠ NIL, when we check [current].info. When is current going to be NIL? When the element is not in the list (and this includes the case when the list is empty).

The correct answer is: The implementation will not work correctly if the element is not in the list, otherwise it works correctly.

Question 10  
Correct  
Mark 1.00 out of 1.00  
Flag question

Consider the following implementation for a function to compute the number of nodes in a singly linked list

```
function size(sll) is:
    count ← 0
    current ← sll.head
    while [current].next ≠ NIL execute:
        count ← count + 1
        current ← [current].next
    end-while
    size ← count
end-function
```

Choose the statement(s) that is/are true about function size.

Select one or more:

- ☐ It is correct.
- ☒ It is not correct, but if we initialize count with 1 it will be correct.
- ☒ It is not correct, but if we change the condition from the while loop to current ≠ NIL, it will be correct.
- ☐ It is not correct, but if we change the condition from the while loop to [[current].next].next ≠ NIL, it will be correct.

Your answer is correct.

The code is not correct. Condition [current].next ≠ NIL, checks if current is the last node. And if it is, the while loop stops and this node will not be counted. So, in general, size returns a value which is the actual size - 1. Initializing count with 1 would solve the problem for most inputs, except the empty lists. Because in case of an empty list current is NIL and the condition [current].next ≠ NIL will crash. So initializing count to 1 is not the solution (or it is, if we check before whether the list is empty and return 0 in that case).

The correct answer is: It is not correct, but if we change the condition from the while loop to current ≠ NIL, it will be correct.

Question 11  
Partially correct  
Mark 0.25 out of 1.00  
Flag question

Assuming that head is a pointer to the first node of a singly linked list, which condition(s) checks if the list is made of exactly 3 nodes? (For a node assume the representation discussed at the lecture). Check multiple solutions if you think that all the conditions from that answer are needed together.

Select one or more:

- ☐ head ≠ NIL
- ☐ [head].next ≠ NIL
- ☐ [[head].next].next ≠ NIL
- ☒ [[[head].next].next].next = NIL
- ☐ head = NIL
- ☐ [head].next = NIL
- ☐ [[head].next].next = NIL
- ☐ [[[head].next].next].next ≠ NIL

Your answer is partially correct.

You have correctly selected 1.

head is the address of the first node.

[head].next is the address of the second node.

[[head].next].next is the address of the third node.

[[[head].next].next].next is the address of the fourth node.

If we want our list to have exactly 3 elements, the address of the fourth node should be NIL. This checks that we do not have more than 3 nodes. But we also have to check that the 3 nodes exists, so head ≠ NIL, [head].next ≠ NIL, [[head].next].next ≠ NIL.

The correct answers are: head ≠ NIL, [head].next ≠ NIL, [[head].next].next ≠ NIL, [[[head].next].next].next = NIL

Question 12  
Incorrect  
Mark 1.00 out of 1.00  
Flag question

Consider the following implementation that adds a new node immediately after the first node of a singly linked list.

```
subalgorithm addSecond(sll, elem) is:
    if sll.head ≠ NIL then
        newNode ← allocate()
        [newNode].info ← elem
        [sll.head].next ← newNode
        [newNode].next ← [sll.head].next
    end-if
end-subalgorithm
```

Pick the statements that are true about the addSecond subalgorithm.

Select one or more:

- ☐ It is correct.
- ☐ If the list is empty, it will crash.
- ☒ If the list has one single element, it will work correctly.
- ☒ If we swap the last two instructions (ignoring end-if and end-subalgorithm), it will work correctly.
- ☐ The code never crashes.

Your answer is incorrect.

It might help if you try to draw this example.

Adding a new element after the first node makes no sense if the list is empty, so this is why we check this condition at the beginning and if the list is empty, we do nothing.

The code is not correct, the problem is with the order of the last two instructions: first we have to set the next of newNode, only after that can we set the next of the head.

What we do now, is a loop. In the last statement, when we set [newNode].next to be [sll.head].next, we make it to point to itself, so we make a cycle.

The correct answers are: If we swap the last two instructions (ignoring end-if and end-subalgorithm), it will work correctly., The code never crashes.

Question 13  
Correct  
Mark 1.00 out of 1.00  
Flag question

We have a singly linked list made of 10 nodes, each node contains 4 bytes of information and a pointer which occupies 4 bytes as well. The first node is at the memory address 1048. What is the address of the last node?

Select one or more:

- ☐ 1128
- ☐ 1058
- ☐ 1120
- ☒ We have no way of knowing

Your answer is correct.

Linked list nodes contain pointers because we have no way of computing where a node is (like we did for a dynamic array).

The correct answer is: We have no way of knowing

Question 14  
Correct  
Mark 1.00 out of 1.00  
Flag question

Which of the following operations has a better complexity for a dynamic array than for a singly linked list?

Select one or more:

- ☐ search for an element
- ☒ return an element from a given position
- ☒ check if it is empty
- ☒ remove the last element
- ☐ find the position of a given element

Your answer is correct.

return an element from a given position and remove the last element have a  $\Theta(1)$  complexity in case of a dynamic array and  $\Theta(n)$  and  $\Theta(n)$  complexity for a singly linked list.

The correct answers are: return an element from a given position, remove the last element

Finish review

◀ Lecture 3 Quiz

Jump to...

Lecture 5 Quiz ▶

Contact site support

You are logged in as Corina Giurgiu (Log out)

IE - DSA

Data retention summary